



Claude Code · SONSLAB

Claude Code 한글 가이드북

처음부터 실전까지 · 한 권으로 끝내는 큐레이션 가이드

발행일	2026-04-26
버전	v2026.17-guidebook
출처	https://code.claude.com/docs
갱신 주기	매주 일요일 20:00 KST
발행처	SONSLAB · shonsubong.github.io/sonslab-blog

SONSLAB 로고는 무엇이든 답을 수 있는 비어 있는 손수레에서 모티브를 가져왔습니다.

들어가며

이 책은 Anthropic이 만든 터미널·IDE 통합 코딩 에이전트 Claude Code를 한국어로 처음부터 끝까지 안내합니다. Anthropic 공식 문서를 단순히 번역해 옮긴 게 아니라, 실제로 매일 쓰면서 부딪치게 되는 순서대로 다시 묶었습니다.

이 책의 원칙

- 독자를 먼저 생각합니다. 실무자가 다음에 무엇을 해야 하는지를 가장 먼저 보여줍니다.
- 가장 쓸모 있는 것을 위로. 설치 다음 페이지부터 바로 첫 명령을 칠 수 있도록 구성했습니다.
- 설명보다 예시. 모든 개념은 실제 명령·코드 한 토막과 함께 등장합니다.
- 최신 상태 유지. 매주 일요일 20:00 KST에 자동 갱신됩니다.
- 중복하지 말고 연결. 공식 문서를 베끼지 않고, 필요한 페이지로 링크합니다.

어떻게 읽으면 좋을까

처음 Claude Code 를 만진다면 Part 1 → Part 2 순서대로 한 번 훑은 뒤, 자기 프로젝트에서 한 시간 정도 가지고 놀아 보세요. 이미 쓰고 있다면 Part 3 (도구 확장) 와 Part 5 (운영) 만 골라 봐도 됩니다. 각 챕터 끝의 “더 알아보기” 박스가 공식 문서의 정확한 페이지로 안내합니다.

주의 · 본 가이드북은 SONSLAB이 정리한 비공식 문서입니다. 권한·보안·결제·법적 사안은 반드시 code.claude.com/docs 의 공식 문서를 1차 자료로 확인하세요.

목차

Part 1 — 시작하기

- 1.1 Claude Code 란 무엇인가
- 1.2 설치와 첫 실행
- 1.3 첫 5분 — 실전 워크플로
- 1.4 Claude Code 가 일하는 방식 (the agent loop)

Part 2 — 일상 사용

- 2.1 CLAUDE.md — 프로젝트 가이드
- 2.2 메모리 — # 와 /memory
- 2.3 슬래시 명령
- 2.4 파일 편집 워크플로
- 2.5 터미널 통합

Part 3 — 도구 확장

- 3.1 MCP — 외부 도구 연결
- 3.2 Skills — 특정 작업 모범 사례
- 3.3 Plugins — 묶음 설치
- 3.4 Hooks — 자동 실행 후크
- 3.5 Subagents — 서브에이전트

Part 4 — 고급

- 4.1 Agent SDK 개요
- 4.2 세션과 체크포인트
- 4.3 Web · IDE · Chrome 환경
- 4.4 Channels — 외부에서 이벤트 밀어넣기

Part 5 — 운영

- 5.1 베스트 프랙티스
- 5.2 보안과 권한
- 5.3 비용 관리
- 5.4 트러블슈팅

부록

- 6.1 자주 쓰는 명령 빠른 참조
- 6.2 한·영 용어집
- 6.3 더 알아보기

PART 1

시작하기

이 책의 첫 다섯 페이지는 "Claude Code가 무엇이고 왜 쓰는가"에 대답하고, 그 다음 다섯 페이지에서 실제로 첫 명령을 칠 수 있도록 합니다. 이 Part를 끝내는 즉시 작은 프로젝트 하나에 Claude Code를 붙여 보세요. 한 시간 정도 직접 만져 본 사람과 그렇지 않은 사람의 이해도는 다섯 배 차이가 납니다.

Chapter 1.1

Claude Code 란 무엇인가

Claude Code 는 Anthropic 이 만든 터미널·IDE 통합 코딩 에이전트입니다. 한 줄로 요약하면 "내 컴퓨터의 셸과 파일 시스템에 직접 손을 댈 수 있는 Claude" 입니다. 챗봇과 다른 점은 두 가지입니다.

첫째, 파일을 직접 편집합니다. 변경 사항을 텍스트로 출력해 사용자에게 복사·붙여넣기를 시키지 않습니다. `apps/web/src/auth.ts:42` 같은 정확한 위치를 짚어 패치를 적용하고, 적용 결과를 다시 읽어 본인 의도와 일치하는지 확인합니다.

둘째, 명령을 실행합니다. `npm test` 를 돌려 결과를 보고, 실패한 테스트 출력을 읽어 다음 수정을 결정합니다. 이 "쓰기 → 실행 → 결과 확인 → 다시 쓰기" 의 반복이 Claude Code 의 핵심 루프이고, 이 루프를 agent loop 이라고 부릅니다.

세 번째 특징을 굳이 더한다면 장시간 작업을 분할해 진행한다는 점입니다. "이 마이그레이션 끝까지 부탁한다" 같은 큰 요청을 여러 단계의 todo 로 쪼개고, 각 단계 결과를 확인해 가며 끝까지 진행합니다.

요약하면 Claude Code 는 "API 한 번 호출 = 한 마디 응답" 의 전통적인 LLM 사용법이 아니라, 여러 차례의 도구 호출과 결과 합성을 거쳐 하나의 작업 단위를 끝내는 에이전트입니다.

누구에게 적합한가

- 프로젝트의 코드 베이스가 큰 경우: 함수 이름·파일 위치를 일일이 알려주는 대신 "이 동작을 추가해 달라" 정도의 요구로 충분합니다.
- 반복 작업이 잦은 경우: 테스트·린트·빌드·배포 명령이 손에 익었다면 Claude 가 그 위에서 작동합니다.
- 혼자 작업하는 사이드 프로젝트: 페어 프로그래머 한 명이 합류한 효과가 가장 큽니다.

적합하지 않은 경우

- 회사 보안 정책으로 외부 LLM 호출이 금지된 환경. 자세한 내용은 Part 5 의 "보안·권한" 챕터에서 다룹니다.
- 창의성·디자인이 80% 이상인 작업. 시각 디자인, 카피라이팅, 사용자 리서치 같은 것은 다른 도구가 더 낫습니다.
- 읽기 전용 검토만 필요한 경우. 그건 그냥 chat 인터페이스가 더 빠릅니다.

Chapter 1.2

설치와 첫 실행

시스템 요구

- macOS, Linux, Windows (WSL 권장) 모두 지원합니다.
- Node.js 18 이상.
- Anthropic 계정 또는 Bedrock / Vertex / Foundry 자격 증명. 개인이라면 Claude Pro 또는 Max 구독으로 시작합니다.

설치

```
npm install -g @anthropic-ai/claude-code
```

-g 플래그는 전역 설치를 의미합니다. **claude** 명령이 PATH 어디서나 실행되도록 합니다.

설치가 끝나면 버전을 확인하세요.

```
claude --version
```

버전이 나오면 끝입니다. **command not found** 가 나온다면 npm 의 글로벌 bin 디렉터리가 PATH 에 들어 있는지 확인해야 합니다 (**npm prefix -g** 로 위치를 확인).

인증

처음 실행할 때 인증 흐름이 시작됩니다. 가장 흔한 경로 두 가지:

```
# 1) Claude Pro / Max 구독 (개인)
claude
# → 브라우저가 열리고 SSO 로그인을 마치면 토큰이 로컬에 저장됩니다.
```

```
# 2) API 키
export ANTHROPIC_API_KEY=sk-ant-xxxxx
claude
```

인증이 한 번 끝나면 다음 실행부터는 자동으로 토큰이 갱신됩니다. 토큰 저장 위치는 macOS 의 경우 **~/ .claude** 디렉터리입니다.

첫 명령

설치한 폴더 또는 아무 프로젝트 루트에서 **claude** 한 줄을 입력하면 대화형 세션이 열립니다. 안으로 들어가면 시각적으로는 평범한 터미널이지만, 그 안에서 자연어로 부탁하면 Claude 가 파일을 편집하고 명령을 돌립니다.

> Explain what this repo does, in three short paragraphs.

이 한 줄로 Claude 는 README, package.json, 핵심 진입점을 스스로 골라 읽고 요약해 줍니다. 처음 만나는 코드 베이스에 적응할 때 가장 빠른 길이고, 동시에 Claude 의 동작 패턴 — "여러 파일을 자기가 골라 읽고 종합한다" — 을 한눈에 보여 주는 예시입니다.

Chapter 1.3

첫 5분 — 실전 워크플로

설치가 끝났다면 다음 다섯 명령을 직접 쳐 보세요. 5분 안에 Claude Code 의 사용법이 손에 들어옵니다.

1) 프로젝트 이해

> What does this repo do? List the top 5 important files.

2) 작은 변경

> Add a /healthz endpoint to the API that returns {ok: true}. Wire it into the existing router.

Claude 는 라우터 파일을 찾아 추가 위치를 정하고, 기존 핸들러 패턴을 따라 새 핸들러를 작성한 뒤 다시 보여 줍니다.

3) 테스트 실행

> Run the existing tests to make sure nothing broke.

테스트가 실패하면 Claude 가 출력을 직접 읽고 원인을 추정해 다시 수정합니다.

4) 커밋 메시지

> Stage the changes and write a one-line commit message in the imperative mood.

5) 다음 일감

> What did we just change? Anything we should clean up before the PR?

이 다섯 단계가 일상 워크플로의 기본 골격입니다. 처음에는 명령을 하나씩 실행해 결과를 확인하고, 익숙해지면 "이 기능 끝까지 부탁" 같은 큰 단위로 묶어 던지게 됩니다.

Chapter 1.4

Claude Code 가 일하는 방식 (the agent loop)

작업을 던지면 Claude 는 내부적으로 다음 다섯 단계를 반복합니다.

- 사용자 요청을 이해합니다. 모호하면 명확화 질문을 하나만 던집니다.
- 계획을 세웁니다. 큰 작업이면 todo 리스트로 분할해 사용자에게 먼저 보여줍니다.
- 도구를 호출합니다. 파일 읽기, 검색, 편집, 셸 실행, 웹 fetch 등.
- 결과를 합성합니다. 도구 출력을 읽고 다음 단계를 결정합니다.
- 결과를 보고하거나 다시 1단계로 돌아갑니다.

이 다섯 단계 루프를 agent loop 이라고 부르며, Claude Code 가 다른 단발 챗봇과 가장 다른 점입니다. 사용자는 "한 마디 → 한 답" 이 아니라 "한 마디 → 여러 도구 호출 → 한 종합 결과" 를 받게 됩니다.

참고 — agent loop 의 구현 세부와 SDK 에서 직접 제어하는 방법은 Part 4 의 "Agent SDK 개요" 챕터에서 다룹니다.

PART 2

일상 사용

Part 1 을 끝낸 사람은 이미 Claude Code 의 기본기를 손에 쥔 상태입니다. Part 2 는 "매일 쓰면서 자연스럽게 마주치는 다섯 가지 기능" 을 다룹니다. 이걸 익히고 나면 90%의 작업을 Claude Code 안에서 끝낼 수 있습니다.

Chapter 2.1

CLAUDE.md — 프로젝트 가이드

CLAUDE.md 는 리포지토리 루트에 두는 한 페이지 분량의 프로젝트 안내문입니다. Claude Code 는 모든 세션 시작 시 이 파일을 자동으로 읽어 컨텍스트로 사용합니다. 다른 사람의 코드 베이스를 이해할 때 사용자가 가장 먼저 읽는 README 와 같은 역할이라고 보면 됩니다.

좋은 **CLAUDE.md** 의 구성 요소:

프로젝트 한 줄 설명

이 서비스가 무엇이고 누구를 위한 것인가.

디렉터리 구조

- `apps/web/` — Next.js 프론트엔드
- `apps/api/` — Fastify 백엔드
- `packages/db/` — Prisma 모델 + 마이그레이션

자주 쓰는 명령

- `pnpm dev` — 전체 개발 서버 띄우기
- `pnpm test` — 단위 테스트
- `pnpm e2e` — Playwright e2e

컨벤션

- 모든 PR 은 한 문장 영문 제목 + 본문 한국어로 작성한다.
- 함수 이름은 동사로 시작.
- 데이터베이스 마이그레이션은 별도 PR 로 분리.

CLAUDE.md 는 길게 쓰지 마세요. 한 페이지가 넘어가면 Claude 는 마지막 절반을 자주 잊습니다. 진짜 자주 참조해야 하는 정보 — 명령, 컨벤션, 절대 건드리면 안 되는 것 — 만 짧게 적어 두는 게 좋습니다.

글로벌 가이드

`~/claude/CLAUDE.md` 에 쓴 내용은 모든 프로젝트에 공통으로 적용됩니다. "응답은 한국어로", "코드 변경 후 항상 테스트 실행" 같은 본인 작업 스타일을 적어 두면 매번 반복할 필요가 없습니다.

Chapter 2.2

메모리 — # 와 /memory

Claude 는 세션이 끝나면 컨텍스트가 사라집니다. 다음 세션에 또 같은 설명을 반복하기 싫다면 메모리 를 활용하세요.

> # 이 프로젝트의 데이터베이스는 PostgreSQL 16 이고, 테스트에서는 Testcontainers 로 띄운다.

한 글자로 시작한 줄은 메모리로 저장됩니다. 다음 세션에 자동으로 다시 주입되어 같은 설명을 반복하지 않아도 됩니다.

저장된 메모리 보기·정리:

> /memory

메모리는 두 종류입니다:

- Project memory (`./.claude/memory/`) — 이 리포에서만 유효. 팀과 공유.
- User memory (`~/.claude/memory/`) — 모든 프로젝트에 유효. 본인만 사용.

저장할 때 어느 쪽에 넣을지 Claude 가 묻거나 합리적으로 추정합니다. 원치 않으면 직접 폴더로 옮기면 됩니다.

무엇을 메모리에 저장할 것인가

좋은 메모리는 "코드를 보면 자명하지 않은 사실"입니다.

- ☒ "이 프로젝트의 인증 토큰은 1시간 만료. 갱신은 cron 이 처리."
- ☒ "결제 모듈은 절대 수정 금지. 외부 PG 인증 통과 코드라 변경 시 재심사 필요."
- ☒ "이 함수는 user 를 받아 string 을 반환한다." (코드를 읽으면 자명)

Chapter 2.3

슬래시 명령

`/` 로 시작하는 짧은 명령으로 자주 쓰는 동작을 빠르게 호출할 수 있습니다.

내장 슬래시 명령 일부:

- `/help` — 사용 가능한 명령 목록
- `/clear` — 현재 세션 컨텍스트 비우기
- `/compact` — 컨텍스트를 요약해 토큰 절약
- `/memory` — 메모리 관리
- `/cost` — 이번 세션 누적 토큰·비용
- `/login, /logout` — 계정 전환
- `/model` — 사용 모델 변경 (Sonnet, Opus, Haiku)
- `/init` — 현재 폴더에 `CLAUDE.md` 자동 생성

사용자 정의 슬래시 명령

`./.claude/commands/<name>.md` 또는 `~/.claude/commands/<name>.md` 파일을 만들면 새 명령이 자동 등록됩니다.

```
---
description: PR 본문 초안 작성
---
```

```
마지막 커밋 5개를 분석해 PR 본문을 작성해 주세요.
- 한국어 제목 (한 문장, 명령어형)
- 본문은 ## 변경 사항 / ## 검증 / ## 영향 받는 부분 세 섹션
```

저장 후 `/draft-pr` 처럼 호출할 수 있습니다. 팀 공통 명령은 프로젝트의 `.claude/commands/` 에 두고 git 으로 공유하면 됩니다.

Chapter 2.4

파일 편집 워크플로

Claude Code 가 코드 편집할 때 따르는 패턴:

- 읽기 먼저. 편집 전에 항상 해당 파일을 다시 읽어 현재 상태를 확인합니다.
- 최소 변경. 의도에 필요한 부분만 손댁니다. 무관한 코드 스타일 변경을 끼워 넣지 않습니다.
- 확인 — 출력. 적용 후 결과 diff 를 사용자에게 보여 줍니다.

이 패턴을 신뢰할 수 있는 이유는 Claude Code 가 **Edit** 같은 정밀한 도구를 사용하기 때문입니다. **old_string** 과 **new_string** 을 정확히 맞추지 않으면 Edit 이 실패합니다. 그래서 "이상한 부분까지 같이 바뀌었네" 같은 사고가 적습니다.

큰 변경은 Plan 모드를 활용하라

여러 파일에 걸친 리팩터링은 변경을 시작하기 전에 계획을 받아 보는 게 좋습니다.

> 먼저 계획부터 세워 주세요. apps/api/src/services 에 있는 모든 svc 클래스를 함수형으로 리팩터링하고 싶습니다.

Claude 는 (1) 어떤 파일을 만질지, (2) 어느 순서로 만질지, (3) 어떤 위험이 있는지를 먼저 보여 줍니다. 마음에 들지 않는 부분은 그 자리에서 수정 요청하고, 동의하면 "그대로 진행" 한 마디로 시작시킵니다.

Chapter 2.5

터미널 통합

Claude Code 는 셸과 한몸처럼 동작하도록 설계했습니다. 다음 패턴을 알아 두면 효율이 크게 오릅니다.

비대화형 모드

스크립트나 CI 에서 Claude 를 한 번 호출하고 싶다면:

```
claude -p "이 디렉터리 안의 .env.sample 을 모든 .env 에 적용해 누락 키를 채워 달라" \
  --output-format json \
  > result.json
```

-p 는 단발 프롬프트, **--output-format** 으로 JSON 으로 받으면 후처리하기 편합니다.

파이프 입력

표준 입력으로 데이터를 흘려 넣을 수도 있습니다.

```
git diff main | claude -p "이 PR 의 위험 요소 5개를 표로 정리해 주세요"
```

이 패턴은 PR 검토, 로그 분석, 에러 메시지 해석 같은 단발 작업에 매우 잘 맞습니다.

셸 사용에 동의 한 번 받기

Claude 가 처음으로 위험할 수 있는 명령 — 파일 삭제, 강제 푸시, 관리자 권한 — 을 시도하면 사용자에게 동의를 묻습니다. 매번 묻지 않게 하려면 **--allow** 옵션이나 슬래시 명령 **/permissions** 로 세션 정책을 조정할 수 있습니다. 보안 측면은 Part 5 에서 자세히 다룹니다.

PART 3

도구 확장

Claude Code 의 능력은 ****얼마나 많은 도구를 손에 쥐고 있느냐****로 결정됩니다. 기본 상태로도 파일 편집·셀 실행·웹 fetch 정도는 되지만, MCP·Skills·Plugins·Hooks·Subagents 다섯 가지를 적절히 끼워 넣으면 능력이 비약적으로 늘어납니다.

Chapter 3.1

MCP — 외부 도구 연결

MCP (Model Context Protocol) 는 Claude Code 에 외부 서비스를 한 단계로 연결하는 표준 인터페이스입니다. 직관적으로는 "Claude 가 사용할 수 있는 USB 포트의 표준" 이라고 보면 됩니다.

대표적인 MCP 서버 예:

- GitHub MCP — 이슈 검색, PR 생성, 코드 리뷰
- Slack MCP — 메시지 검색, 채널 요약
- Linear / Asana / Jira MCP — 티켓 이동, 스프린트 진행
- Postgres MCP — 안전한 SELECT 쿼리 실행
- Filesystem MCP — 특정 폴더 안에서만 파일 조작

추가 방법

MCP 서버 등록은 한 줄 명령으로 끝납니다:

```
claude mcp add github -- npx -y @modelcontextprotocol/server-github
```

설치 후 다음 세션부터 Claude 가 GitHub 의 이슈·PR 도구를 자기가 알아서 호출합니다.

어떤 MCP 부터 깔까

처음엔 다음 셋만 설치하세요. 가장 자주 쓰입니다.

- GitHub — 이슈·PR·코드 검색
- fetch / brave-search — 외부 문서·자료 검색
- filesystem — 작업 폴더 외부의 안전한 디렉터리 접근

나머지는 본인 워크플로에 등장하는 서비스부터 추가하면 됩니다. 한 번에 너무 많은 MCP 를 등록하면 Claude 가 매 호출마다 도구 목록을 검토하느라 응답이 느려질 수 있습니다.

Chapter 3.2

Skills — 특정 작업 모범 사례

Skill 은 "이 작업은 이렇게 처리해라" 라는 모범 사례·가이드 묶음입니다. SKILL.md 파일에 트리거 조건과 절차를 적어 두면, 해당 패턴이 등장할 때 Claude 가 자동으로 그 가이드를 따릅니다.

기본 제공되는 Skills 일부:

- pdf — PDF 읽기·생성·병합·폼 작성
- docx / pptx / xlsx — 오피스 파일 처리
- schedule — 반복 일정 작업 만들기
- skill-creator — 새 스킬 만드는 도우미

직접 만들기

```
mkdir -p ~/.claude/skills/my-pr-flow
```

```
---
name: my-pr-flow
description: 우리 팀 PR 작성 절차. "PR 만들어 줘", "/pr" 또는 사용자가 PR 본문을 부탁할 때 사용.
---
```

우리 팀 PR 절차

1. main 에서 분기한 브랜치인지 확인
2. 변경 파일이 5개 이상이면 사용자에게 의견을 한 번 물을 것
3. 본문은 다음 세 섹션:
 - ## 변경 사항 (왜)
 - ## 검증 방법 (어떻게 확인했는지)
 - ## 위험 (롤백 시점)
4. 한 줄 영문 제목, 본문은 한국어

저장 후부터 "PR 본문 부탁" 같은 요청에 Claude 가 위 절차를 자동 적용합니다.

Chapter 3.3

Plugins — 묶음 설치

Plugin 은 Skills · MCPs · 슬래시 명령을 한 묶음으로 패키징한 단위입니다. 본인이나 팀의 작업 묶음을 한 번에 배포할 때 사용합니다.

설치는 마켓플레이스에서:

```
claude plugin install engineering@knowledge-work-plugins
```

이 한 줄로 engineering 플러그인의 10개 스킬 (architecture, code-review, debug, deploy-checklist, documentation, incident-response, standup, system-design, tech-debt, testing-strategy) 이 한 번에 추가됩니다.

본인 플러그인 만들기

anthropic-skills:skill-creator 또는 **cowork-plugin-management:create-cowork-plugin** 스킬이 가이드해 줍니다. 디렉터리 구조는 다음과 비슷합니다.

```
my-team-plugin/
├── plugin.json
├── skills/
│   ├── pr-flow/SKILL.md
│   └── deploy/SKILL.md
├── commands/
│   └── ship.md
├── mcp/
│   └── servers.json
```

.plugin 확장자로 zip 해 동료에게 공유하거나 사내 마켓에 등록하면 됩니다.

Chapter 3.4

Hooks — 자동 실행 후크

Hook 은 특정 이벤트가 발생했을 때 자동으로 실행되는 사용자 정의 스크립트입니다. 가장 흔한 용도:

- PreToolUse — 위험한 명령 실행 전에 가로채 검증

- PostToolUse — 도구 호출 결과를 가공해 후처리
- Stop — 응답 완료 시 알림 보내기

예시 — **rm -rf /** 류 명령을 무조건 차단:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "if": "Bash(rm -rf /*)",
        "command": "block-rm.sh"
      }
    ]
  }
}
```

if 조건에 맞는 도구 호출이 발생하면 **block-rm.sh** 가 실행되고, 그 스크립트가 **exit 2** 로 종료되면 도구 호출 자체가 중단됩니다.

어디에 두는가

- 글로벌: **~/.claude/settings.json** 의 **hooks** 섹션
- 프로젝트: **./.claude/settings.json**
- 관리자 강제: 회사 PC 의 시스템 정책 위치 (Part 5 보안 챕터 참조)

혹은 코드를 줄이는 게 아니라 위험을 차단하고 정책을 강제하는 용도로 가장 강력합니다.

Chapter 3.5

Subagents — 서브에이전트

Subagent 는 특정 역할에 특화된 보조 Claude 인스턴스입니다. "이 단계에서는 코드 리뷰만 해 줘" 같은 좁은 임무를 맡길 때 씁니다.

내장 서브에이전트 예:

- code-reviewer — diff 를 읽고 보안·성능·정확성 관점으로 리뷰
- explorer — 코드 베이스 탐색·검색에 특화
- plan — 구현 전에 단계별 계획만 작성

직접 만들기

```
---
name: db-migrator
description: 데이터베이스 마이그레이션 전용. 스키마 변경 시 사용.
tools: Read, Edit, Bash
---
```

당신은 PostgreSQL 마이그레이션 전문가입니다. 다음 규칙을 지킵니다:

1. NOT NULL 컬럼 추가 시 항상 백필 마이그레이션 분리
2. 50M 이상 테이블의 ALTER 는 CONCURRENTLY 옵션 우선
3. 모든 변경에 롤백 SQL 첨부

이렇게 만들어 두면 큰 작업 중간에 "마이그레이션 부분은 db-migrator 에게 맡겨" 한 마디로 호출할 수 있습니다. 메인 Claude 의 컨텍스트는 깨끗하게 유지되고, 서브에이전트 영역의 작업 결과만 다시 메인으로 합쳐집니다.

PART 4

고급

Part 3 까지가 "도구를 더 단다" 였다면, Part 4 는 "Claude Code 자체를 프로그래밍한다" 입니다. CLI 안에서가 아니라, 자신의 어플리케이션·CI·서버 안에서 Claude Code 를 부품처럼 호출하고 싶을 때 읽으세요.

Chapter 4.1

Agent SDK 개요

Claude Agent SDK 는 Claude Code 의 agent loop 를 자기 코드 안에서 직접 돌리게 해 주는 라이브러리입니다. 파이썬과 타입스크립트 두 가지 언어를 공식 지원합니다.

가장 짧은 사용 예 (Python):

```
from anthropic_agent import Agent

agent = Agent(
    model="claude-opus-4-6",
    system="당신은 우리 회사의 코드 리뷰어입니다.",
    tools=["read_file", "grep", "edit_file", "run_tests"],
)

result = agent.run("apps/api 디렉터리의 보안 문제 다섯 개를 찾아 표로 정리해 주세요")
print(result.final_message)
```

Agent SDK 는 다음 세 가지를 직접 제어할 수 있게 해 줍니다.

- 시스템 프롬프트 — Claude 의 정체성·규칙을 자기 도메인에 맞게.
- 도구 집합 — 기본 도구 외에 자기 회사 API 도 도구로 등록.
- 승인 흐름 — 위험한 도구 호출 전에 사용자에게 확인받는 정책.

자세한 API 는 공식 문서의 [Agent SDK reference](#) 페이지를 참고하세요. 본 가이드북에서는 "왜 SDK 를 쓰는가" 와 "기본 패턴" 만 다룹니다.

Chapter 4.2

세션과 체크포인트

긴 작업은 도중에 끊어 두었다가 이어서 진행하는 능력이 중요합니다.

세션 저장

대화형 모드에서 진행하던 세션은 자동으로 저장됩니다. 다시 시작하려면:

```
claude --continue      # 가장 최근 세션
claude --resume <session> # 특정 세션 ID
```

체크포인트

긴 리팩터링 같은 작업 중간에 명시적으로 체크포인트를 찍을 수 있습니다.

```
> /checkpoint "DB 마이그레이션 직전 상태"
```

이후 잘못된 방향으로 갔다면 다음 명령으로 되돌립니다.

```
> /restore
```

체크포인트는 코드 변경뿐 아니라 대화 컨텍스트도 함께 보관합니다. git stash 와 비슷하지만 conversation state 까지 함께라는 점이 다릅니다.

Chapter 4.3

Web · IDE · Chrome 환경

Claude Code 는 터미널이 본인이지만, 다른 진입점도 있습니다.

Claude Code on the Web

브라우저에서 Claude Code 를 띄울 수 있습니다. 본인 PC 에서 진행하던 세션을 노트북으로 옮겨 이어가거나, 가벼운 작업을 모바일에서 진행할 때 유용합니다. [cLaude.com/code](https://claude.com/code) 에서 접속.

IDE 통합

VS Code, Cursor, JetBrains, Zed 등 주요 IDE 와 통합됩니다. IDE 안에서 Claude 의 변경 사항이 바로 diff 뷰로 나타나고, "이 변경 거부" / "이 변경만 받기" 를 단축키로 처리할 수 있습니다.

설치는 IDE 마켓플레이스에서 "Claude Code" 검색.

Claude Code in Chrome

브라우저 기반 자동화에 특화된 베타입니다. 웹 폼 채우기, 사이트 검색, 페이지 캡처 같은 일을 Claude 가 직접 합니다. 회사 보안 정책상 Chrome 자동화가 허용된 환경에서만 사용하세요.

Chapter 4.4

Channels — 외부에서 이벤트 밀어넣기

Channel 은 실행 중인 Claude 세션에 외부에서 이벤트를 흘려 보내는 통로입니다. 사용자가 입력하지 않아도 Claude 가 반응하게 만들 수 있습니다.

가장 흔한 사용 예:

- Monitor 도구 — 백그라운드에서 로그 파일을 감시하다가 5xx 가 뜨면 그 자리에서 Claude 에게 알림.
- CI 알림 — 테스트가 실패할 때 결과 페이로드를 채널로 전달, Claude 가 즉시 원인 추정.
- 외부 큐 — Slack 멘션이나 PagerDuty 알림을 Claude 세션의 새 메시지처럼 들어오게.

Channels 는 Agent SDK 와 함께 사용할 때 가장 빛납니다. 단발 스크립트가 아니라 장시간 살아있는 에이전트 워커 를 만들 수 있게 해 주는 핵심 기능입니다.

PART 5

운영

업무에 본격 도입할 때 마주치는 네 가지 — 베스트 프랙티스, 보안과 권한, 비용 관리, 트러블슈팅 — 을 모았습니다.

Chapter 5.1

베스트 프랙티스

이 다섯 가지는 거의 모든 사용자에게 효과가 있습니다.

- 1) **CLAUDE.md** 와 메모리에 진짜로 자주 참조하는 것만 적기. 길어질수록 효과가 떨어집니다. 한 페이지를 절대 넘기지 마세요.
- 2) 큰 작업은 Plan 모드부터. 5분 들여 계획을 받아 보면, 잘못된 방향으로 30분 달려갔다가 되돌리는 사고가 거의 사라집니다.
- 3) 슬래시 명령으로 반복을 줄이기. "이 패턴 또 나오네" 싶으면 5분 들여 슬래시 명령으로 만드세요. 다음부터는 한 글자 입력으로 끝납니다.
- 4) 테스트 먼저. Claude 에게 "기능 X 추가" 를 부탁하기 전에 "X 의 동작을 검증할 테스트를 먼저 써 주세요" 를 부탁하세요. 그 다음 구현을 부탁하면 정확도가 크게 오릅니다.
- 5) 자주 **/cost** 확인. 토큰을 얼마나 쓰고 있는지 인식하지 못하면 어느 순간 청구서가 무겁습니다. 매 세션 끝에 한 번씩 확인하는 습관을 들이세요.

Chapter 5.2

보안과 권한

Claude Code 는 매우 강력한 도구입니다. 그래서 권한 통제가 중요합니다.

사용자 레벨 권한 정책

claude /permissions

명령으로 다음을 제어할 수 있습니다:

- 어떤 셸 명령을 자동 허용할지 (e.g., **git status**)
- 어떤 명령을 매번 확인할지 (e.g., **git push --force**)
- 어떤 명령을 항상 차단할지 (e.g., **rm -rf /**)

관리자 레벨 강제 정책 (Managed Settings)

회사에서 단체로 사용한다면 사용자 설정으로는 부족합니다. Managed Settings 는 관리자가 강제하는 정책으로, 사용자 설정보다 우선합니다.

저장 위치 (Windows):

HKLM\SOFTWARE\Policies\ClaudeCode
C:\Program Files\ClaudeCode\managed-settings.json

전형적인 정책 항목:

- 허용된 MCP 서버 목록 (allowlist)
- 차단된 명령 패턴 (denylist)
- 텔레메트리 / 로그 전송 설정
- 사용 가능 모델 제한

자세한 내용은 공식 문서의 [Set up Claude Code for your organization](#) 페이지를 참고하세요.

절대 하지 말 것

- API 키를 코드에 박아 두지 마세요. `.env` 와 OS 키체인을 사용하세요.
- `--dangerously-skip-permissions` 류 옵션은 일회성 디버깅 외에 절대 쓰지 마세요.
- Claude 에게 "이 폴더 다 지워" 같은 표현보다 "이 파일 목록을 지워" 처럼 명시적으로 부탁하세요.

Chapter 5.3

비용 관리

Claude Code 는 사용량 기반 과금입니다. 비용을 통제하는 가장 효과적인 다섯 가지:

- 모델 선택 — Haiku 는 Sonnet 의 1/5 가격, Opus 는 Sonnet 의 5배. 단순 작업은 Haiku, 어려운 추론은 Opus.
- `/compact` — 긴 대화는 주기적으로 압축. 같은 토큰을 두 번 결제하지 않습니다.
- 세션 분리 — 다른 주제의 작업은 새 세션으로. 컨텍스트가 길수록 매 호출 비용이 커집니다.
- MCP 도구 수 제한 — 도구가 많을수록 매 호출에 도구 정의 토큰이 추가됩니다.
- `/cost` 모니터링 — 매일 한 번 확인하는 습관.

월 한도 알림은 Anthropic Console 에서 설정할 수 있습니다 (console.anthropic.com → Billing → Usage limits).

Chapter 5.4

트러블슈팅

자주 마주치는 문제와 해결.

Q. 응답이 느려졌어요.

→ 등록된 MCP 도구가 너무 많지는 않은지 확인하세요. `claude mcp list` 로 목록을 보고, 한동안 안 쓴 도구는 제거합니다.

Q. "권한이 없습니다" 라는 메시지가 자주 나와요.

→ `/permissions` 로 자주 쓰는 안전한 명령은 자동 허용으로 추가하세요. `git status` 같은 읽기 명령이 매번 묻는 건 불필요한 마찰입니다.

Q. 같은 질문을 다시 물어봐야 하나요?

→ `#` 메모리를 활용하세요 (Section 2.2). 또는 `CLAUDE.md` 에 적어 두면 자동으로 매 세션에 들어갑니다.

Q. 모델을 바꾸고 싶어요.

→ `/model` 명령으로 세션 내에서 즉시 전환됩니다. 영구 변경은 `~/.claude/settings.json` 의 `model` 키.

Q. 마이그레이션 같은 위험한 작업을 자동으로 하지 못하게 하고 싶어요.

→ Hook (Section 3.4) 으로 PreToolUse 차단을 거세요.

Q. 갑자기 인증이 풀렸어요.

→ `claude /login` 다시 실행. 토큰은 보통 30~90일 단위로 만료됩니다.

Q. 응답이 매번 영어로 나옵니다.

→ `~/.claude/CLAUDE.md` 에 "응답은 한국어로" 한 줄을 추가하세요.

PART 6

부록

Chapter 6.1

자주 쓰는 명령 빠른 참조

```
|명령 | 동작 |
|:---|:---|
|`claude` | 대화형 세션 시작 |
|`claude -p "...` | 단발 프롬프트 |
|`claude --continue` | 마지막 세션 이어서 |
|`claude --resume <id>` | 특정 세션 이어서 |
|`claude mcp list` | 등록된 MCP 서버 목록 |
|`claude mcp add <name> -- <cmd>` | MCP 서버 추가 |
|`claude plugin install <id>` | 플러그인 설치 |
|`claude --version` | 버전 확인 |
|`/help` | 슬래시 명령 도움말 |
|`/clear` | 컨텍스트 비우기 |
|`/compact` | 대화 요약·압축 |
|`/memory` | 메모리 관리 |
|`/cost` | 토큰·비용 |
|`/model` | 모델 전환 |
|`/permissions` | 권한 정책 |
|`/init` | CLAUDE.md 자동 생성 |
|`/checkpoint "msg"` | 체크포인트 저장 |
|`/restore` | 체크포인트 복원 |
```

Chapter 6.2

한·영 용어집

```
|English | 한글 | 설명 |
|:---|:---|:---|
|Agent loop | 에이전트 루프 | 입력 → 추론 → 도구 호출 → 결과 합성을 반복하는 핵심 흐름 |
|Subagent | 서브에이전트 | 특정 역할에 특화된 보조 Claude 인스턴스 |
|Hook | 후크 | 특정 이벤트에 자동 실행되는 사용자 정의 스크립트 |
|Slash command | 슬래시 명령 | `/명령` 형태로 호출하는 사용자 정의 / 내장 동작 |
|Skill | 스킬 | 특정 작업의 모범 사례·도구 묶음. SKILL.md 로 정의 |
|Plugin | 플러그인 | 스킬·MCP·명령을 묶어 한 번에 설치하는 패키지 |
|MCP | 모델 컨텍스트 프로토콜 | 외부 도구·서비스를 Claude 에 연결하는 표준 인터페이스 |
|Managed settings | 관리형 설정 | 관리자가 강제하는 정책 설정 (개인 설정보다 우선) |
|Sandboxing | 샌드박스 | Claude Code 가 실행되는 격리 환경 |
|Checkpointing | 체크포인팅 | 대화·세션 상태 스냅샷 저장·복원 |
|Session | 세션 | 하나의 작업 단위. 메시지·도구 호출·결과를 묶음 |
|CLAUDE.md | 프로젝트 가이드 | 리포지토리 루트의 프로젝트 컨벤션 문서 |
|Permission | 권한 | 도구·명령 사용 가능 여부 정책 |
|Bedrock / Vertex / Foundry | AWS / GCP / Azure 통합 | 클라우드 LLM 게이트웨이를 통한 Claude Code 호출 |
|Streaming output | 스트리밍 출력 | 응답을 토큰 단위로 점진 전달 |
|Structured output | 구조화 출력 | JSON 등 정해진 스키마로 응답을 받음 |
|Tool search | 도구 검색 | 수많은 도구 중 작업에 필요한 것만 동적 로드 |
|Channels | 채널 | 실행 중인 세션에 외부 이벤트를 밀어 넣는 통로 |
|Plan mode | 계획 모드 | 변경 전에 단계별 계획만 먼저 받는 모드 |
```

Chapter 6.3

더 알아보기

이 가이드북은 큐레이션된 입문서입니다. 더 깊은 자료가 필요하면 다음을 보세요.

- 공식 문서 — <https://code.claude.com/docs> · 가장 정확하고 최신.
- 공식 변경 노트 — <https://code.claude.com/docs/changelog> · 매주 업데이트.
- Agent SDK 참조 — <https://code.claude.com/docs/agent-sdk/python> · 직접 프로그래밍.

- Anthropic Cookbook — <https://github.com/anthropics/anthropic-cookbook> · 검증된 예제.
- SONSLAB Claude Code 팁 — <https://shonsubong.github.io/sonslab-blog/claude-tips/> · 매일 짧은 한국어 팁.

질문·오류 제보는 SONSLAB 블로그의 About 페이지에 있는 이메일로 보내 주세요. 다음 갱신에 반영됩니다.

— 본 PDF 는 2026-04-26 (KST) 기준으로 자동 생성되었습니다. SONSLAB —